

ROS2 · rclpy · FastAPI · React

ROS2 Communication Monitor Dashboard

실시간 통신 모니터링과 Interface Lab

ROS2 상태를 한눈에 보고, 필요한 통신을 직접 실행하는 웹 대시보드



MONITOR

관찰

+

INTERFACE LAB

직접 실행

기능 중심 발표 자료

보이지 않던 ROS2 통신을 화면으로 확인합니다

목록·상태·관계를 자동으로 모으고, 등록된 Interface로 실제 통신도 시험합니다.

① 상태 확인

Node, Topic, Service, Action의
현재 존재 여부와 통신 상태를 봅니다.

② 관계 이해

어떤 Node가 발행·구독하고
요청·응답하는지 연결 관계를 봅니다.

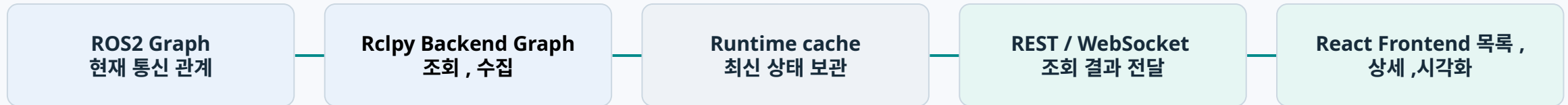
③ 직접 실행

Interface를 등록한 뒤 Publish, Call,
Goal 실행 결과와 history를 확인합니다.

Node = 기능을 수행하는 실행 단위 · Topic = 계속 전달되는 채널 · Service = 요청 1회 + 응답 1회 · Action = 진행 상황이 있는 작업

ROS2 Graph 상태가 웹 화면까지 전달되는 구조

Backend가 ROS2 Graph를 주기적으로 수집하고, cache를 거쳐 Frontend 화면에 출력합니다.



왜 cache를 쓰나?

화면 요청마다 ROS2를 다시 스캔하지 않고 이미 모은 최신 상태를 빠르게 반환합니다.

REST의 역할

Topic·Service·Action·Node의 상세 목록과 snapshot을 전달합니다.

WebSocket의 역할

연결 상태와 Alert count 같은 가벼운 요약 정보를 보조 전달합니다.

사용 기술 스택과 사용 목적

Backend와 Frontend에 사용한 핵심 기술과 각 기술의 역할을 정리했습니다.

구분	기술 스택
Backend	 ROS2  GRAPH API  RCLPY  FASTAPI  WEBSOCKET
Frontend	 REACT  VITE  REACT FLOW

기술별 사용 목적



ROS2

Topic, Service, Action, Node 로 구성된
로봇 통신 환경 플랫폼



GRAPH API

실행중인 ROS2리소스와 연결관계를 조회



RCLPY

Python 환경에서 ROS2 Node, Subscription 구성



FASTAPI

수집된 ROS2 상태와 기능을 Front에 제공



YAMI

Message 타입과 주요 Interface 외부 설정으로 분리



WEBSOCKET

연결 상태와 Alert 요약을 실시간 전달하는 통신채널



REACT

ROS2 상태를 실시간 반영하기 위한 동적 UI 구성



VITE

React 개발 환경 단순화 및 빠른 화면 수정 과 실행



REACT FLOW

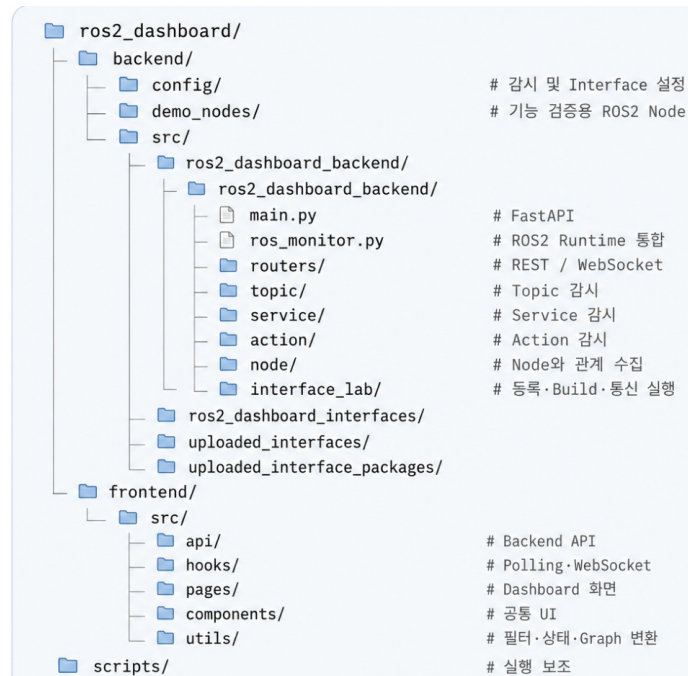
Node와 통신 관계를 확대,이동 가능한 그래프로 시각화

프로젝트 폴더 구조

Backend는 ROS2 상태 수집과 실행 API를, Frontend는 화면 표시와 관계 시각화를 담당합니다.

핵심 폴더 구조

생성물과 로그 폴더는 제외



backend/config

모니터링 대상, Interface 등록 상태, Timeout 정책을 보관

backend/src

FastAPI, rclpy Runtime, Topic·Service·Action·Node 감시 로직

interface_lab

Interface 등록·업로드
· Build·Publish·Receive·Call·Goal 실행

frontend/src

REST/WebSocket 데이터를 받아 표, 상세, Alert, Graph 화면 구성

데이터 흐름

ROS2 Graph → rclpy Runtime → Cache → API → React

핵심: backend는 ROS2와 API를 담당하고, frontend는 API 결과를 화면과 그래프로 표현합니다.

Overview는 전체 상태와 Alert를 한눈에 보여줍니다

요약 카드는 상태를 넓게 보여주고, Alert는 조치가 필요한 조건만 선별합니다.



Node · Topic · Service · Action 요약

최근 Alert와 상태 분포

Badge와 Alert는 별도 기준

Alert는 현재 장애와 최근 해결된 장애를 구분합니다

장애가 지속되는 동안 active로 표시하고, 해결된 Alert는 60초간 resolved로 남겨 원인 확인 시간을 확보합니다.

현재 Alert

현재 장애가 남아 있는 항목입니다.
warning/error 집계에 포함되며 즉시 확인해야 합니다.

이전 Alert

이미 해결된 항목입니다.
현재 장애 집계에서는 제외하고 60초 동안만 남깁니다.

정책 요약

active는 장애가 지속되는 동안 유지
resolved는 60초 동안 보관
재발 시 기존 Alert를 active로 전환

장애 발생 중: 현재 Alert

Alert					
현재 장애와 최근 해결된 Alert를 구분해 표시합니다					
현재 Alert 2			이전 Alert 6		
레벨	출처	이름	메시지	코드	감지 시각
주의	topic	/demo_cleaning_schedule	Subscriber exists but no publisher is available.	waiting_publisher	오전 9:39:58
오류	node	/demo_cleaning_schedule_to_pic_publisher	Node connection lost; it is no longer visible in the ROS2 graph.	node_stale	오전 9:39:58

해결 후: 이전 Alert

Alert					
현재 장애와 최근 해결된 Alert를 구분해 표시합니다					
현재 Alert 0			이전 Alert 8		
레벨	출처	이름	메시지	코드	해결 시각
해결됨	topic	/demo_cleaning_schedule	Topic message has not been received within stale timeout.	topic_stale	오전 9:40:28
해결됨	node	/demo_cleaning_schedule_topic_publisher	Node connection lost; it is no longer visible in the ROS2 graph.	node_stale	오전 9:40:27

Topic은 latest · Hz · stale을 실시간으로 확인합니다

Backend가 지원 타입을 자동 구독하고, 수신 callback으로 최신값과 수신 속도를 갱신합니다.

Topic 알림 없음

Topic 상세

기본 화면은 현재 활동 중이거나 최근 상태 변화가 관찰된 Topic만 표시합니다.

Topic 이름 또는 타입 검색

숨김 포함 주요 항목 전체 대기 중 정상 주의 오류 미수신 구독자 없음 미지원

구독자 없음은 현재 해당 Topic을 받는 외부 Node가 없다는 뜻입니다. 센서 출력, 로그, 이벤트를 Topic에서는 장애가 아닐 수 있습니다.

상태	이름	타입	발행	구독	외부 구독	Hz	상세 감시	마지막 값	마지막 확인
정상	/joint_states	sensor_msgs/msg/JointState	1	2	1	961.00 Hz	예	{"name":["wheel_left_joint","wheel_r...}	오후 4:11:52
구독자 없음	/imu	sensor_msgs/msg/Imu	1	1	0	199.20 Hz	예	{"orientation":{"x":0.002875349381415...	오후 4:11:52
정상	/odom	nav_msgs/msg/Odometry	1	3	2	50.00 Hz	예	{"position":{"x":-1.3074432045573203,...}	오후 4:11:52
정상	/cmd_vel	geometry_msgs/msg/TwistStamped	3	2	1	30.00 Hz	예	{"linear":{"x":0.29999999999999993,"y...	오후 4:11:52
정상	/cmd_vel_smoothed	geometry_msgs/msg/TwistStamped	1	2	1	20.00 Hz	예	{"linear":{"x":0.29999999999999993,"y...	오후 4:11:52
정상	/cmd_vel_nav	geometry_msgs/msg/TwistStamped	6	2	1	10.00 Hz	예	{"linear":{"x":0.29999999999999993,"y...	오후 4:11:52
정상	/scan	sensor_msgs/msg/LaserScan	1	8	7	5.00 Hz	예	{"angle_min":0,"angle_max":6.28000020...	오후 4:11:52

이름 /cmd_vel_teleop

타입 geometry_msgs/msg/TwistStamped

상태 waiting_publisher

상태 이유 subscriber exists but no publisher

마지막 갱신 오후 4:11:52

연결 정보

연결 Node

실행/측정 정보

상세 데이터

장치 상태 값

원본 Preview JSON

Publisher / Subscriber 수

Hz와 최신 데이터

발행자 대기 · 미수신 상태

Service는 목록 수집과 실제 호출 가능 여부를 분리합니다

존재 여부를 자동으로 관찰하고, 응답 시간과 결과는 요청이 있을 때만 발생하며 사용자가 호출했을 때 기록됩니다.

Service 이름, 타입, 상태 검색

주요 항목

응답 측정

대기/오류

전체

내부/관리 포함

상태	이름	타입	분류	서버	클라이언트	응답 측정	호출 가능	마지막 호출	마지막 요청	마지막 응답	응답 시간	마지막 측정	슬깍
서버 대기	/fromLL	robot_localization/srv/FromLL	사용자	0	1	상태만 표시	아니오	-	-	-	-	-	아니오
정상	/RobotControl	rths_interfaces/srv/RobotControl	사용자	1	1	마지막 응답 있음	예	5분 전	{"cmd":5}	{"success":true,"result_code":0,"mess...	6.14 ms	-	아니오
정상	/ScheduleCrud	rths_interfaces/srv/ScheduleCrud	사용자	1	1	마지막 응답 있음	예	5분 전	{"cmd":4,"table_name":"","items":[],"...	{"success":true,"message":"demo Sched...	3.11 ms	-	아니오

상태 요약

이름

/RobotControl

타입

rths_interfaces/srv/RobotControl

분류

user

기본 슬깍

아니오

상태

active

상태 이유

service server available

마지막 갱신

오후 4:12:02

연결 정보

Server / Client 관계

호출 가능 여부

마지막 요청·응답과 응답 시간

Action은 Goal · Feedback · Result 상태를 관찰합니다

모니터링은 새 Goal을 만들지 않고, 관찰된 Goal의 진행과 종료 상태를 화면에 보여줍니다.

전체 Action
18

실패/취소
0

활동 Action
4

주의/오류
0

실행 중
0

관찰 Goal
315

성공
4

Action Alert

Action 알림 없음

Action 상세

Action 목록은 3초마다 갱신됩니다. Goal 전송과 cancel은 제공하지 않습니다.

Action 이름, 타입, 상태 검색

대기 Action 포함 **주요 항목** 전체 실행 중 성공 실패/취소 Goal 미관찰

상태	이름	타입	서버	클라이언트	마지막 GOAL	실행 가능	GOAL 전송	마지막 GOAL	피드백	결과	실행 시간	관찰 GOAL
정상	/CanControl	rths_interfaces/action/CanControl	1	1	응답 성공	예	4분 전	{"node_id":5,"port":0,"value":0,"retr...	수신됨	성공	500.26 ms	1
정상	/compute_path_to_pose	nav2_msgs/action/ComputePathToPose	1	3	성공	아니오	-	-	수신 없음	성공	0.57 ms	153
정상	/follow_path	nav2_msgs/action/FollowPath	1	3	성공	아니오	-	-	수신됨	성공	291.47 ms	154
정상	/navigate_to_pose	nav2_msgs/action/NavigateToPose	1	8	성공	아니오	-	-	수신됨	성공	15621.83 ms	7

Action 상세

정상

/compute_path_to_pose

nav2_msgs/action/ComputePathToPose

결과 조회 정책: 관찰된 Goal만 조회합니다. 대시보드는 Goal을 직접 보내지 않고, 상태 topic에서 관찰한 Goal이 종료되었을 때만 결과를 조회합니다.

상태 요약

이름 /compute_path_to_pose

타입 nav2_msgs/action/ComputePathToPose

상태 active

상태 이유 action server available

마지막 갱신 오후 4:12:07

연결 정보

실행/측정 정보

상세 데이터

마지막 Goal JSON

Goal 상태

Feedback 수신

Result와 실행 시간

Node 화면은 실행 단위의 통신 참여 관계를 보여줍니다

Node 하나를 선택하면 발행·구독 Topic, 응답·요청 Service, Action 관계를 함께 확인합니다.

상태	NODE	네임스페이스	발행	구독	응답 SERVICE	요청 SERVICE	GOAL 실행	GOAL 요청	마지막 확인
실행 중	/demo_can_control_server	/	4	0	10	0	1	0	2s 전
실행 중	/demo_schedule_crud_service	/	2	0	8	0	0	0	2s 전
실행 중	/ros2_dashboard_topic_monitor	/	2	2	7	4	0	1	2s 전

1. Node 목록: 참여 수 요약

목록에서 보는 값

- 발행 Topic 수
- 구독 Topic 수
- 응답/요청 Service 수
- Goal 실행/요청 수

Node 이름만이 아니라 통신 참여 관계를 함께 봅니다.

2. Node 상세정보

Node 상세

/ros2_dashboard_topic_monitor

현재 ROS2 Graph에서 발견된 Node입니다.

상태 요약

전체 이름 /ros2_dashboard_topic_monitor

이름 ros2_dashboard_topic_monitor

네임스페이스 /

상태 active

상태 이유 node discovered in ROS2 graph

마지막 확인 0s 전

마지막 경신 오전 11:49:49

3. Topic 관계

연결 정보 · Topic

발행자 수 2

구독자 수 2

발행 TOPIC

/parameter_events
rcl_interfaces/msg/ParameterEvent

/rosout
rcl_interfaces/msg/Log

구독 TOPIC

/CanControl/_action/feedback
rths_interfaces/action/CanControl_FeedbackMessage

/CanControl/_action/status
action_msgs/msg/GoalStatusArray

4. Service 관계

연결 정보 · Service

응답 Service 7

요청 Service 4

응답 SERVICE

/ros2_dashboard_topic_monitor/describe_parameters
rcl_interfaces/srv/DescribeParameters

/ros2_dashboard_topic_monitor/get_parameter_types
rcl_interfaces/srv/GetParameterTypes

/ros2_dashboard_topic_monitor/get_parameters
rcl_interfaces/srv/GetParameters

더 보기 4개

요청 SERVICE

/CanControl/_action/cancel_goal
action_msgs/srv/CancelGoal

/CanControl/_action/get_result
rths_interfaces/action/CanControl_GetResult

/CanControl/_action/send_goal
rths_interfaces/action/CanControl_SendGoal

더 보기 1개

핵심 포인트

1. 현재 active 상태 확인
2. Topic·Service 연결 수 확인
3. 상세 패널에서 실제 이름과 타입 확인

사라진 Node는 timeout 동안 stale로 남겨 화면 흔들림을 줄입니다.

Visualization은 Node 중심의 통신 관계를 그래프로 보여줍니다

기존 REST 데이터를 Frontend에서 조합해 React Flow nodes와 edges로 변환합니다.



선택 Node 중심 1-hop

Topic · Service · Action 연결

전체 Graph와 배치 초기화

자동 모니터링은 관찰, Interface Lab은 사용자 실행입니다

두 영역은 목적과 실행 시점, 저장하는 결과가 다릅니다.

AUTOMATIC MONITORING

현재 ROS2 시스템을 자동으로 관찰

- Graph discovery
- 상태·수·latest·Hz·stale
- Runtime cache와 snapshot
- 새 Call / Goal 자동 실행 안 함

INTERFACE LAB

등록한 타입으로 통신을 직접 실행

- Interface 등록·업로드
- Build / Apply / import 확인
- Publish · Receive · Call · Goal
- 결과와 history 확인

관찰 결과를 보고



필요할 때 실행

Interface는 직접 작성하거나 업로드로 등록할 수 있습니다

간단한 정의는 화면에서 작성하고, 실제 장비 타입은 원본 패키지 단위로 등록합니다.

UPLOAD / APPLY / RUN
인터페이스 작업 도구

타입 기입 타입 업로드 Package zip 업로드 Package 폴더 업로드 replace 적용하기 등록 목록

타입 직접 등록 인터페이스 직접 작성

.msg/srv/action 파일을 uploaded_interfaces 패키지에 직접 생성합니다. 저장 전 문법 검증을 수행하며, 저장 후 적용하기 build가 필요합니다.

저장 위치: backend/src/uploaded_interfaces/srv/MyControl.srv

kind

srv

type name

MyControl

definition

```
uint8 cmd
---
bool success
string message
```

문법 검증 인터페이스 저장

타입 등록, 빌드 적용, Service/Action 테스트

타입 등록은 "사용자" 단위로 등록합니다.

단일 타입 등록만으로도

최소(C) 파일 열기

최근

홈

바탕 화면

다운로드

문서

비디오

사진

음악

+ 다른 위치

hs 다운로드 rths_interfaces

이름	크기	종류	수정
src			18 12월 2025
action			7월 15일
srv			7월 15일
include			7월 15일
msg			7월 15일
package.xml	921 바이트	Markup	18 12월 2025
CMakeLists.txt	1.1 kB	Text	3월 6일

타입 직접 등록

이미 빌드/source 되어 있음

full type

rths_interfaces

description

전체 Message

☐ 읽기 전용 디렉터리 열기

kind · type name 입력

msg / srv / action 정의 작성

문법 검증 후 저장

Apply는 패키지 빌드와 import 확인까지 포함합니다

업로드된 ROS2 Interface 패키지를 빌드하고 Backend가 실제 Python 타입을 import할 수 있는지 확인합니다.

Uploaded Interface Packages

목록보기

장비가 실제 사용하는 원본 interface package를 패키지명 그대로 등록합니다.

rths_interfaces

import됨

path backend/src/uploaded_interface_packages/rths_interfaces

source uploaded_package

uploaded_a2026-07-21T04:15:09.407270+00:00

삭제

msg 1

rths_interfaces/msg/CleaningSchedule

import됨

fields

uint32 scheduling_id

string scheduling_dt

uint8 count

bool is_active

srv 2

rths_interfaces/srv/RobotControl

import됨

request

uint8 cmd

response

bool success

uint8 result_code

string message

rths_interfaces/srv/ScheduleCrud

import됨

request

uint8 cmd

string table_name

rths_interfaces/CleaningSchedule[] items

패키지 경로와 source 확인

msg · srv 타입 import됨

실제 장비 타입 확장 가능

Topic Lab은 같은 타입으로 Publish와 Receive를 시험합니다

채널 이름, Message full type, payload를 입력하고 송수신 결과를 history에서 확인합니다.

PUBLISH

Topic name 데이터를 보낼 채널

Full type 사용할 Message 구조

Payload Message에 넣을 실제 값

Publish

payload 변환 → ROS2 발행 → history

RECEIVE

Receive
start

메시지
수신

history
저장

Receive
stop

사용자가 시작한 구독만 유지하며, stop 시 subscription을 정리합니다.

Frontend 입력 → topic execution router → InterfaceReceiveRuntime → value converter → ROS2 → history

Service Call과 Action Goal은 사용자가 명시적으로 실행합니다

full type으로 Goal schema를 만들고, 입력값을 ROS2 객체로 변환해 실행 결과를 history에 저장합니다.

UPLOAD / APPLY / RUN

인터페이스 작업 도구

타입 가입

타입 업로드

Package zip 업로드

Package 폴더 업로드

☐ replace

적용하기

등록 목록

Package 목록

Topic 실행

Service 실행

Action 실행

수신

/CanControl Action 수신 관찰을 시작했습니다.

등록 Action 실행

목표보기

☒ Action import됨만 보기 1/1

Action · 1/1개

import됨 · 호출 가능 · /CanControl · rths_interfaces/action/CanControl

호출 가능

선택 타입 rths_interfaces/action/CanControl의 Goal schema 5개 필드로 폼을 생성합니다.

node_id uint8

1

port uint8

2

value uint32

2

retries uint8

4

timeout_ms uint32

5

timeout_sec

10

Goal 실행

수신

목표보기

Topic 수신

Service 수신

Action 수신

Mock 준비중

항목 검색

Action 이름 또는 type 검색

Action · 1/1

/CanControl · rths_interfaces/action/CanControl

수신 중

수신 중지

선택 이력 리셋

전체 이력 리셋

새로고침

Action feedback/result receive history · 5개

/CanControl · feedback

```
{
  "stage": "sending",
  "attempt": 1,
  "detail": "demo send attempt 1/4"
}
```

/CanControl · feedback

```
{
  "stage": "sending",
  "attempt": 2,
  "detail": "demo send attempt 2/4"
}
```

/CanControl · feedback

Action Goal 입력 폼

Feedback / Result 수신 영역

history로 실행 결과 확인

CASE 1 · 실제 장비 Interface 패키지 의존성

타입 정의뿐 아니라 원본 패키지 구조와 의존성까지 있어야 build · import · 통신이 가능했습니다.

문제

실제 장비의 ROS2 Interface를 받았지만, 대시보드 환경에는 해당 패키지가 없어 정확한 Service·Action 타입을 불러올 수 없었습니다.

원인

사용자 정의 Interface는 타입 이름만으로 사용할 수 없습니다. msg/srv/action 정의와 package.xml 의존성, CMakeLists.txt 빌드 설정, 원본 패키지명이 함께 필요했습니다.

해결

장비별 Interface 패키지를 ZIP/폴더 단위로 업로드하고, build/apply와 import-check를 거쳐 Backend가 실제 타입을 import하도록 구성했습니다.

결과

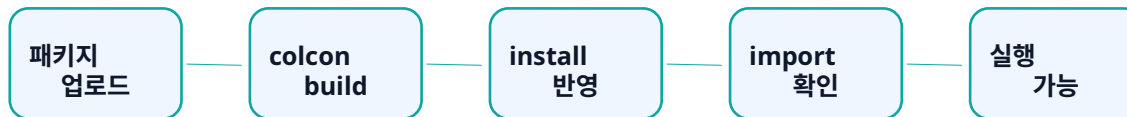
Service·Action 타입 인식, Feedback/Result 해석, 장비별 패키지 확장이 가능해졌습니다.

```
rths_interfaces/srv/ScheduleCrud
import됨

request
uint8 cmd
string table_name
rths_interfaces/CleaningSchedule[] items
bool only_active
string where
string options

response
bool success
string message
rths_interfaces/CleaningSchedule[] items
```

실제 화면: Uploaded Interface Packages / rths_interfaces / msg·srv import됨



핵심: .msg/.srv/.action은 타입 정의이고, package.xml과 CMakeLists.txt는 의존성·빌드 방법을 담습니다.

CASE 2 · FastAPI와 ROS2 spin 실행 충돌 방지

웹 서버 요청 처리와 ROS2 callback 처리를 동시에 유지하기 위해 spin을 별도 thread로 분리했습니다.

문제

FastAPI는 REST/WebSocket 요청을 계속 처리해야 하고, ROS2는 spin으로 timer와 callback을 계속 처리해야 했습니다.

원인

두 작업은 모두 계속 실행되는 blocking 성격이 있어, 같은 실행 흐름에서 순서대로 실행하면 한쪽이 다른 쪽을 막을 수 있었습니다.

해결

FastAPI lifespan에서 RosMonitor를 시작하고, rclpy.spin()은 daemon thread에서 실행하도록 분리했습니다.

결과

웹 요청 처리와 ROS2 메시지 수신·Graph 갱신을 동시에 유지하고, 서버 종료 시 ROS2 자원도 함께 정리했습니다.

BEFORE



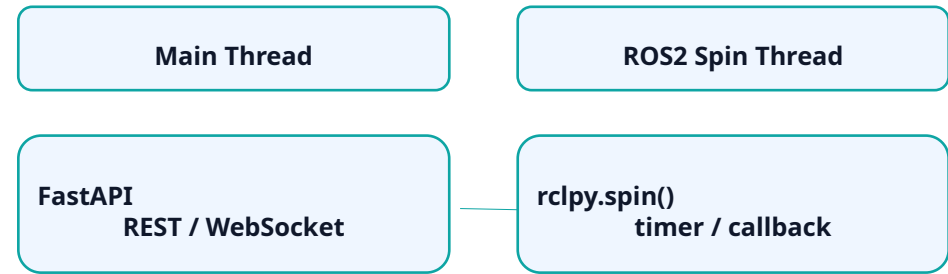
blocking

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    """ROS2 Dashboard Backend에서 요청된 처리를 수행하는 함수입니다."""
    ros_monitor.start()
    try:
        yield
    finally:
        ros_monitor.stop()

app = FastAPI(lifespan=lifespan)
  
```

AFTER



동시에 실행

```

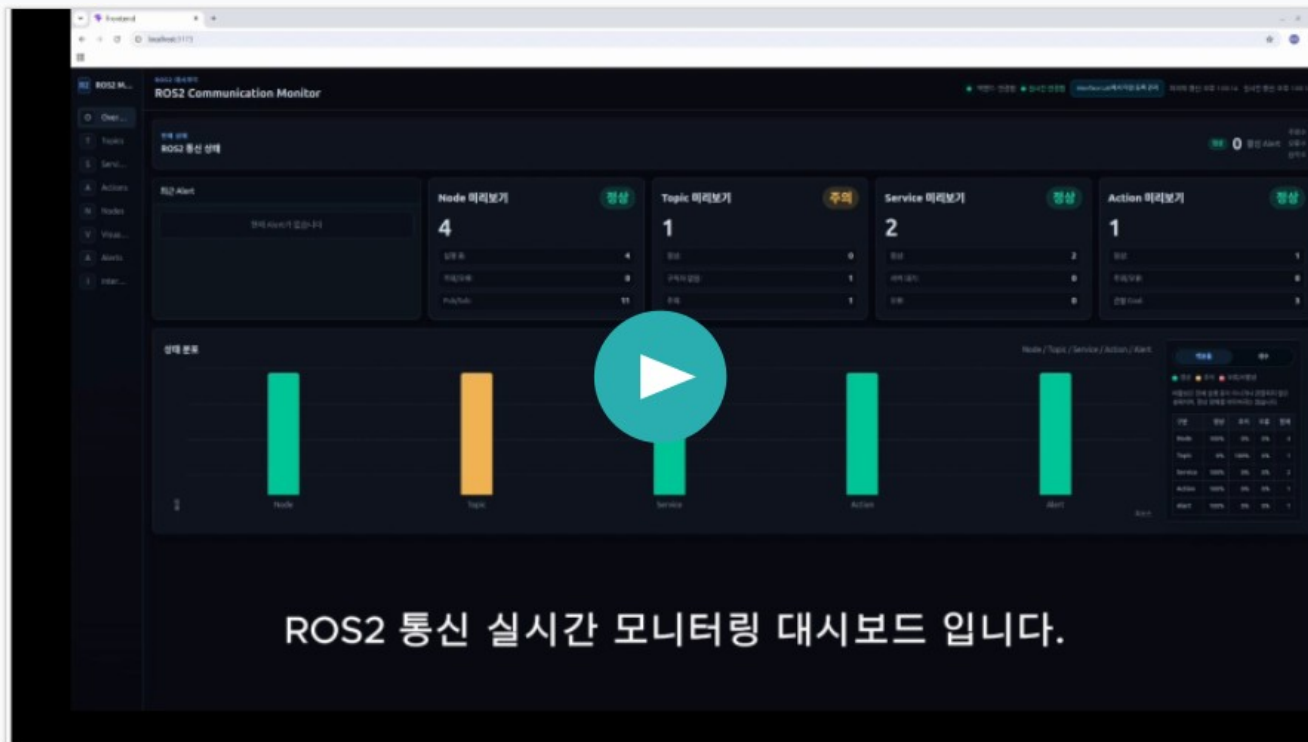
def start(self) -> None:
    """RosMonitor coordinator에서 요청된 처리를 수행하는 함수입니다."""
    if self._thread and self._thread.is_alive():
        return

    rclpy.init(args=None)
    self._node = Node('ros2_dashboard_topic_monitor')
    self._node.create_timer(
        self._config.poll_interval_sec,
        self._update_graph,
    )
    self._update_graph()

    self._thread = Thread(target=self._spin, daemon=True)
    self._thread.start()
  
```

시연 영상

실제 UI 기능 검수 영상으로, 자동 모니터링과 Interface Lab 실행 흐름을 확인합니다.



시연에서 확인할 흐름

화면을 따라가며 “관찰 → 실행 → 경고 확인”을 순서대로 보여줍니다.

- 관찰**
 Overview · Topic · Service · Action · Node
 상태 확인
- 실행**
 Interface Lab에서 Publish · Receive · Call ·
 Goal 실행
- 검증**
 Alert, Visualization, history로 결과 확인

핵심: 시연 영상은 실제 UI에서 ROS2 통신 상태를 확인하고, 등록 Interface로 직접 통신을 실행하는 과정을 보여줍니다.

확장 개발

현재 구현된 Monitoring / Interface Lab을 기반으로 현장 장비화와 지능형 진단까지 확장합니다.

1. 현장형 엣지 모니터링 장비

Jetson Nano · Raspberry Pi와 터치 디스플레이에 대시보드를 탑재해, 현장에서 ROS2 연결 상태와 통신 이상을 바로 확인하는 독립형 진단 장비로 확장합니다.

2. LLM 기반 장애 원인 추정

통신 로그, Alert, Node 상태, Service Timeout, Action 실패 결과를 LLM API 또는 Local LLM과 연결해 원인 후보와 우선 점검 항목을 제안합니다.

3. 주요 통신 자동 선별

YAML 필터 조건, Graph 관계, Alert 발생 빈도, 통신 의존도를 함께 분석해 우선 확인해야 할 Topic · Service · Action · Node를 자동 선별합니다.

4. 통신 유형별 관리

향후 통신 명세의 목적, 발행 조건, 예상 주기와 통신 관계를 기준으로 생존 통신, 상태 센서 Stream, 명령, Event 통신을 구분 하여 통신 특성에 맞는 관리가 가능하도록 개선합니다.

최종 목표: ROS2 통신을 보여주는 대시보드를 넘어, 현장에서 상태 확인 · 장애 분석 · 우선순위 판단까지 지원하는 지능형 진단 플랫폼으로 확장합니다.